

08 — Cheap Edge Compute for GPs

What you'll learn: the full menu of techniques for running GPs cheaply, their trade-offs, and a decision table for "what runs on the vehicle" vs "what we use ground-side."

Prereqs: 03 (the $O(n^3)$ problem),
05 (Kalman duality),
06 (GMRF/sparse precision).

Why it matters here: the eventual target is on-vehicle, GPU-free compute.

1. The problem restated

Exact GP inference is $O(n^3)$ time, $O(n^2)$ memory (doc 03).

On a vehicle CPU with no GPU, hard real-time, and a power budget, that's a non-starter for large n .

The good news: for the structured problems we have (temporal, or spatial-with-locality), there are

exact linear-time methods — we don't even need approximations for the first build.

2. The menu (cheapest path first for our case)

(a) Sliding window — trivial, used for the demo

Keep only the last W seconds of DVL data ($n \approx 60\text{--}180$ at ~ 1 Hz). $O(n^3)$ on $n \approx 100$ is microseconds.

Refit hyperparameters every `refit_every` samples, not every sample. This alone makes the demo

real-time-trivial. Limitation: forgets old data (fine — velocity dynamics are local).

(b) State-space / Kalman GP — exact, $O(n)$, the edge winner for temporal

A Matérn temporal GP **is** a Kalman filter (doc 05):

- $O(n)$ time, $O(1)$ memory per step (recursive), no matrix to store.
- Exact (not approximate) for Matérn kernels.
- Naturally streaming and asynchronous; closed-form interpolation.
- Maps onto imu-drift's `no_std` Rust → literally embeddable.

This is the recommended edge form for the velocity model.

(c) GMRF / SPDE sparse precision — exact-ish, $O(n)$ - $O(n^{\{3/2\}})$ for spatial

For spatial/spatio-temporal fields (the current model), the SPDE→GMRF construction gives a sparse

precision matrix solved with sparse Cholesky (doc 06).

Scales where dense GPs can't; the spatial counterpart of (b).

(d) Inducing-point / sparse approximations — for big, unstructured data

When data is large and lacks Markov structure, summarize it with $m \ll n$ **inducing points**:

- **FITC/SoR/DTC** (Snelson & Ghahramani; Quiñonero-Candela & Rasmussen): $O(n m^2)$.
- **SVGP / variational sparse GPs** (Titsias; Hensman): mini-batch trainable, $O(m^3)$ per step, the

modern default in GPyTorch for scale.

Approximate (quality depends on m and inducing-point placement). Probably unnecessary for us given

(b), but the standard tool if structure is absent.

(e) Structured-kernel tricks — for gridded data

KISS-GP / SKI (Wilson & Nickisch) exploit Kronecker/Toeplitz structure on grids for near-linear cost. Niche for us (our data isn't gridded) but powerful where it applies.

(f) Implementation-level levers (orthogonal to the above)

- **Single precision (float32)** — $\sim 2\times$ speed/memory; usually fine for navigation.
- **Fixed-lag smoothing** — smooth only the last L steps for bounded latency/cost.
- **Quantization / fixed-point** — for microcontroller targets, on the Kalman recursion.
- **Precompute & freeze hyperparameters** — learn θ offline from logs, run inference-only on-board

(hyperparameters drift slowly; no need to optimize on the vehicle).

- **Reuse the Cholesky factor** across mean/variance/marginal-likelihood (already standard).

3. Decision table — our project

Component	Where	Method	Cost	Deps
DVL-velocity GP — **demo**	ground-side / dev	sliding-window exact GP (GPyTorch)	$O(n^3)$, $n \approx 100 \rightarrow$ trivial	torch/gpytorch
DVL-velocity GP — **edge**	on vehicle	**state-space / Kalman GP** (method b)	**$O(n)$** , $O(1)$ mem	numpy or 'no_std' Rust
Ocean-current field (future)	on vehicle / ground	**SPDE-GMRF** (method c), Markov-in-time	$O(n) - O(n^{\{3/2\}})$	scipy.sparse / FEM
Hyperparameter learning	offline	marginal-likelihood + Adam, then **freeze**	one-off	torch/gpytorch

Punchline: we get the best of both worlds with **no accuracy compromise** — develop and tune with

the flexible GPyTorch batch GP now, then deploy the mathematically equivalent $O(n)$ Kalman form on the

vehicle, with hyperparameters frozen from logs (or taken from imu-drift's Allan-variance fit).

4. What we deliberately avoid

- No GPU dependency on-vehicle (the edge form is CPU/microcontroller-friendly).
- No giant unstructured dense GP (we always have temporal or spatial structure to exploit).
- No online hyperparameter optimization on-board (freeze them; they move slowly).

5. Sizing intuition

- State-space Matérn-3/2 velocity GP: state ≈ 2 per axis $\times 3$ axes $\approx 6-9$ floats; per-step cost is a

handful of tiny matrix ops — **negligible** next to the existing 16-state EKF.

- Sliding-window batch GP at $n=120$, 3 outputs (multitask 360×360): a few ms per refit on a laptop —

fine for the demo, refit every few seconds.

- A 2-D current GMRF with $\sim 10^3-10^4$ mesh nodes: seconds per solve with sparse Cholesky

ground-side;
needs the state-space-in-time trick for on-board real-time.

Key takeaways

- For **temporal** GPs (our velocity model), the **state-space/Kalman** form is **exact and $O(n)$** —

the edge solution, no approximation needed.

- For **spatial** fields (the current model), **SPDE/GMRF sparse precision** is the analogue.
- **Inducing points (SVGP/FITC)** are the fallback only when there's no structure — we have structure,

so we likely won't need them.

- Strategy: tune flexibly off-board (GPyTorch), deploy the equivalent $O(n)$ form on-board with frozen

hyperparameters. Same accuracy, tiny cost.

Further reading

- Quiñero-Candela & Rasmussen (2005), unifying view of sparse approximate GPs.
- Hensman, Fusi, Lawrence (2013), Gaussian Processes for Big Data (SVGP).
- Wilson & Nickisch (2015), KISS-GP.
- Särkkä, Solin, Hartikainen (2013), *Spatiotemporal learning via infinite-dimensional Bayesian

filtering and smoothing* (state-space GPs).

- Next: 09 — Glossary and annotated references.